# Memory Matters: Evaluating Learning Behavior in LLM-Based Agents

## I. Introduction

Large Language Models (LLMs) have demonstrated strong capabilities in generation, reasoning, and few-shot learning [1], [2]. Building on these capabilities, LLM-based agent systems support tasks that require reasoning, action, tool use, and interaction with external environments [3], [4]. These systems are increasingly considered for organizational workflows that involve decision-making, tool use, and coordinated action across multiple steps [5]. In such workflows, memory allows agents to retain task-relevant information from previous interactions, such as decisions, feedback, tool outputs, and constraints, and reuse it when the workflow continues [6].

Recent evaluations show that current agents still struggle to maintain and reuse task-relevant information across multi-session interactions [6]. A related challenge is that LLM-based agents often maintain task information through the current prompt and the interaction history that fits within the context window, which becomes limiting in long conversations and extended tasks [7]. Reliable memory use therefore requires more than storing information: the agent must retrieve relevant information and apply it correctly when the workflow continues [6].

To address these limitations, recent work has introduced memory-augmented agents that store and retrieve information beyond the immediate prompt [7]. MemGPT proposes virtual context management, where information is moved between the limited LLM context window and external memory stores [7]. Reflexion uses feedback and self-reflection to maintain task-specific experience across trials [8]. Retrieval-augmented generation combines parametric models with non-parametric memory accessed through retrieval [9]. GraphRAG extends retrieval-augmented generation by constructing a graph-based text index over source documents [10]. Together, these approaches show different ways of supporting memory in agentic systems, including context management, reflection, retrieval, and structured representation.

**Context and Challenges (Etteplan):** This study is motivated by an academic–industrial collaboration with Etteplan[1], where researchers and Etteplan engineers jointly design and develop a memory-enabled agentic system. The industrial context involves iterative Design for Excellence (DFx) workflows, where engineering decisions, feedback, constraints, and task goals can evolve across interactions. In such workflows, the challenge is not only to generate a correct response for the current prompt, but also to preserve and reuse relevant information from earlier interactions. This creates a need to study memory mechanisms in realistic engineering tasks without exposing company-specific data or proprietary rules.

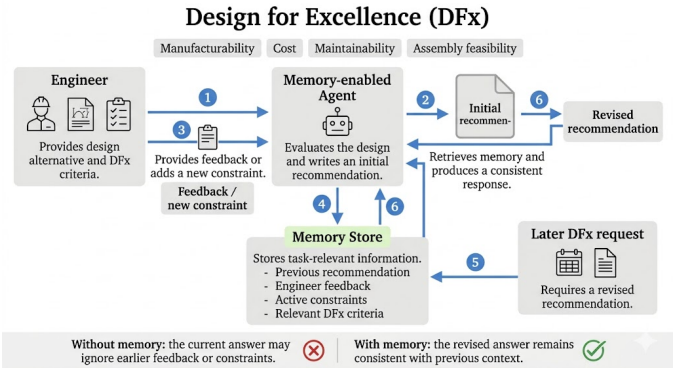



Fig. 1. An example of memory use in an iterative Design for Excellence (DFx) workflow. The figure shows how stored recommendations, engineer feedback, active constraints, and relevant DFx criteria support consistent revised recommendations across interactions.

**Example:** Figure 1 illustrates a representative DFx workflow in an industrial engineering setting.[2] An agent evaluates a design alternative using given criteria and produces an initial recommendation. The engineer then provides feedback or introduces an additional constraint, such as cost, material, maintainability, or assembly feasibility. When the workflow continues, the agent must reuse the earlier recommendation, feedback, and active constraints. If this information is not retained and retrieved correctly, the agent may produce a response that fits the current prompt but conflicts with the previous workflow context.

Despite recent progress in memory-augmented agents, it remains difficult to compare memory mechanisms under common conditions. Existing approaches are often evaluated with different tasks, agent pipelines, memory interfaces, and performance measures. As a result, it is unclear whether observed improvements come from the memory mechanism, the underlying LLM, the task design, or the surrounding agent pipeline.

This work aims to design and evaluate a memory-centered agentic framework for iterative engineering workflows. The goal is not to propose a new memory mechanism, but to integrate and evaluate existing memory mechanisms within a

[1]https://www.etteplan.com/

[2]The example is illustrative and abstracted from the industrial context. It does not disclose Etteplan-specific project data, customer information, proprietary design rules, or confidential engineering processes.

common agentic pipeline. We study how these mechanisms influence agent behavior across repeated and interdependent tasks, and how their effects vary across LLM backbones and memory-use patterns. The evaluation combines public memory-oriented benchmarks with industrially motivated DFx scenarios developed through the academic–industrial collaboration.

We formulate the following research questions (RQs).

*RQ1: How do different memory mechanisms influence agent performance and learning behavior in iterative, interdependent tasks?*

**Rationale:** Different memory mechanisms use different strategies to store, retrieve, and update past information. The answer to RQ1 helps us understand how these mechanisms affect agent performance and learning behavior under comparable task settings.

*RQ2: To what extent do memory-augmented agents improve over repeated interactions and transfer learned knowledge across related tasks compared with stateless agents?*

**Rationale:** Memory is expected to support improvement over time and reuse of prior knowledge. The answer to RQ2 evaluates whether memory-augmented agents improve and transfer knowledge compared with stateless baselines.

*RQ3: How does the choice of LLM backbone affect the effectiveness of memory-based learning?*

**Rationale:** The usefulness of memory may depend on the reasoning capability and instruction-following behavior of the underlying LLM. The answer to RQ3 helps explain whether different models vary in how effectively they use memory for adaptation and decision-making.

*RQ4: How effectively do agents retrieve and use stored memory during task execution?*

**Rationale:** Performance improvements may depend not only on storing memory, but also on retrieving and applying it correctly. The answer to RQ4 evaluates memory-use quality in terms of relevance, correctness, and impact on task decisions.

**Contributions:** This study makes the following contributions:

- **Contribution 1:** ....
- **Contribution 2:** ....
- **Contribution 3:** ....

**Paper organization:** The remainder of this paper is organized as follows. Section **??** presents the background and related work on LLM-based agents, memory mechanisms, and agent-memory evaluation. Section III describes the research methodology, including the agentic framework, memory mechanisms, task settings, models, and evaluation metrics. Section **??** presents the results of the study. Section **??** discusses the findings, implications, and lessons learned from the benchmark and industrial DFx evaluations. Section **??** reports threats to validity. Finally, Section **??** concludes the paper and outlines directions for future work.

## II. Related Work

### A. Memory Mechanisms for LLM-Based Agents

LLM-based agents rely on memory mechanisms to retain and reuse information across interactions. Existing work has explored this problem from several directions, including long-term conversational memory, external memory stores, reflective memory, structured memory, and memory management. MemoryBank introduces a long-term memory mechanism that stores, recalls, updates, and selectively forgets user-related information to support sustained interaction with LLM-based assistants [11]. MemGPT proposes virtual context management, where information is moved between the limited context window and external memory stores to support longer interactions [7]. Reflexion stores verbal feedback and self-reflection from previous trials, allowing agents to reuse past experience in later attempts [8].

Other studies focus on more structured or adaptive memory mechanisms. A-MEM proposes an agentic memory system that organizes memories as interconnected notes and updates their links as new experiences are added [12]. MIRIX introduces a multi-agent memory system with several memory types, including episodic, semantic, procedural, and resource memory [13]. Memory-R1 studies memory management as a learned process, where agents decide when to add, update, delete, or ignore memory entries using reinforcement learning [14]. In embodied settings, HELPER uses an external memory of language-program pairs to retrieve relevant examples for future task planning and to personalize interactions over time [15]. These studies show that memory can be implemented through different mechanisms, but they are often evaluated in different agent pipelines, task settings, and interaction contexts.

### B. Benchmarks and Evaluation of Agent Memory

Several benchmarks have been proposed to evaluate memory in LLMs and LLM-based agents. LoCoMo evaluates very long-term conversational memory using multi-session dialogues and tasks such as question answering, event summarization, and multimodal dialogue generation [16]. LongMemEval evaluates long-term memory in chat assistants through information extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention [17]. MemBench extends memory evaluation by considering factual and reflective memory across participation and observation scenarios, and evaluates memory from effectiveness, efficiency, and capacity perspectives [18]. MemoryAgentBench evaluates memory agents through incremental multi-turn interactions and focuses on accurate retrieval, test-time learning, long-range understanding, and selective forgetting [19].

Recent benchmarks also move toward more agentic and interdependent settings. The LTM benchmark evaluates long-term memory and continual learning through a single extended conversation where multiple tasks are interleaved [20]. MemoryArena evaluates memory in multi-session agentic tasks where agents must learn from earlier actions and feedback and use that memory to guide later actions [6]. These benchmarks are important because they move beyond static recall and evaluate whether memory is useful for later decisions. However, existing benchmarks still differ in their tasks, agents, memory

interfaces, and evaluation metrics, which makes it difficult to compare memory mechanisms under common conditions.

### C. Conclusive Summary

Reviewing related work suggests that memory is becoming a central component of LLM-based agents. Existing studies have proposed different memory mechanisms, including external memory stores, reflective memory, structured memory, multi-type memory, and learned memory management. Existing benchmarks have also improved the evaluation of long-term memory by moving from static recall tasks toward multi-session and agentic settings. However, limited work has compared different memory mechanisms within a common agentic pipeline while also considering iterative engineering workflows. In particular, it remains unclear how memory mechanisms influence agent performance, learning behavior, consistency, and memory use when the same agentic framework is evaluated across both public benchmarks and industrially motivated DFx scenarios. In contrast to existing work, this study integrates existing memory mechanisms into a shared agentic framework and evaluates their behavior in repeated, interdependent tasks, including benchmark tasks and DFx scenarios developed through an academic–industrial collaboration.

## III. Research Methodology

This section describes the research methodology used to evaluate memory mechanisms in LLM-based agentic systems. The study does not propose a new memory mechanism or a new agentic framework. Instead, we develop an agentic system using the Microsoft Agent Framework [21], [22] and use it as the implementation platform for integrating and comparing existing memory mechanisms. The study follows a controlled factorial experimental design [23] in which memory systems, LLM backbones, and iteration depth are varied while the task environment and evaluation procedure are kept consistent.

### A. Study Design

The study evaluates how different memory mechanisms affect agent behavior in iterative and interdependent tasks. We use a multi-session setting in which agents solve a sequence of related subtasks. Each subtask can introduce new objectives, constraints, or feedback, while later subtasks may depend on information introduced earlier. This setting allows us to observe whether agents can retain, retrieve, and apply task-relevant information across interactions.

The experimental setup varies two main independent variables: the memory system and the LLM backbone. A third factor, iteration depth, captures how agent behavior changes across sequential interaction rounds. The same task sequences, agent pipeline, and evaluation procedure are used across configurations where possible. This design supports comparison between memory-enabled agents and a stateless baseline, as well as comparison across different memory mechanisms and model families.

### B. Agentic System (Etteplan)

The agentic system is implemented using the Microsoft Agent Framework [21], [22]. The framework is used to orchestrate the agent workflow, connect LLM backbones, integrate memory mechanisms, and execute iterative tasks. We use the framework as an implementation platform and not as a research contribution.

At a high level, the system consists of a task interface, an agent controller, an LLM interface, a memory component, and a logging component. For each task, the agent receives the current input, optionally retrieves relevant memory, generates an output using the selected LLM backbone, receives evaluation feedback, and updates memory depending on the memory configuration. The logging component records task inputs, agent outputs, retrieved memories, memory updates, feedback, and evaluation scores. These traces are used to analyze both task performance and memory-use behavior.

The final implementation details of this subsection will be completed with input from the industrial partner, including the specific Microsoft Agent Framework setup, memory integration points, supported tools or resources, storage backend, and logging mechanism.

### C. Memory Systems

To evaluate the role of memory in agent behavior, we include a stateless baseline and a set of memory-enabled systems. The selected systems represent different memory design strategies, including reflection-based learning, retrieval-based memory, managed long-term memory, skill-based memory, and structured temporal memory. Table I summarizes the memory systems considered in this study.

For each memory system, we record the information written to memory, the memory entries retrieved during task execution, and the extent to which retrieved information is used in the final response. This allows us to evaluate memory not only through final task scores, but also through the quality of memory retrieval and use.

### D. LLM Backbones

To examine whether memory effectiveness depends on the underlying model, we evaluate the selected memory systems with LLMs from different providers and deployment settings. The planned model families include OpenAI, Google, Anthropic, Meta, and DeepSeek models. Table II summarizes the candidate LLM backbones considered in this study.

These models represent variation in reasoning capability, instruction following, context handling, deployment model, and computational cost. Including both cloud-based and local or open-weight models allows us to examine whether memory mechanisms behave differently across model families and deployment settings. Within each comparison group, we keep model settings as consistent as possible, including temperature, maximum output length, system prompts, and task instructions. Any model-specific differences, such as context-window size, tool-use support, pricing, or deployment constraints, will be documented during the analysis. The final model versions

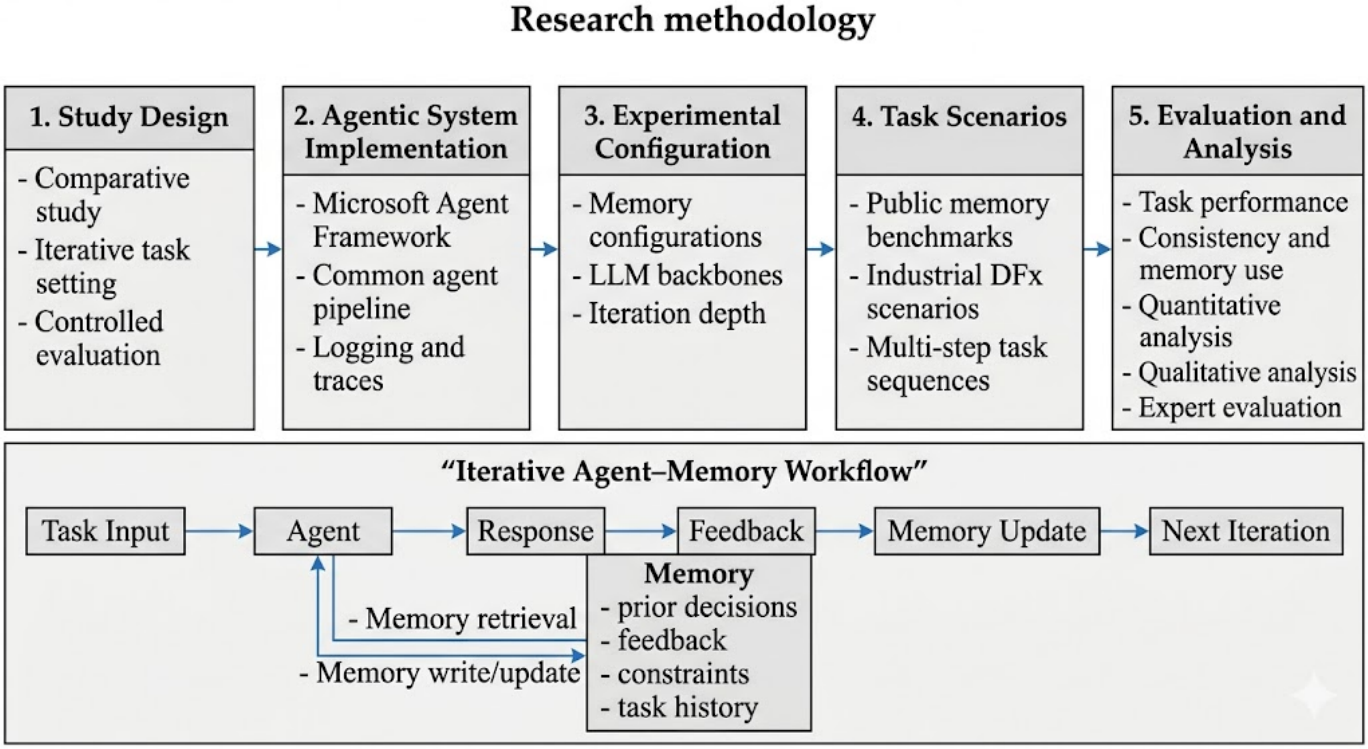



Fig. 2. Overview of the research methodology. The study evaluates memory-enabled agentic systems across different configurations, task scenarios, and iterative interactions.

will be selected based on availability, API access, context-window requirements, and reproducibility constraints at the time of experimentation.

### *E. Task Settings*

The evaluation uses two task settings: public memory-oriented benchmarks and industrially motivated DFx scenarios.

**Public memory-oriented benchmarks.** Public benchmarks are used to evaluate memory mechanisms in controlled and reproducible settings. These benchmarks include tasks that require agents to recall information, update stored knowledge, reason across sessions, or use prior feedback. Each benchmark is adapted to the agentic system while preserving its original evaluation goal.

**Industrially motivated DFx scenarios.** The second task setting is based on an industrial Design for Excellence (DFx) case provided through the collaboration with Etteplan. Each DFx scenario is modeled as a sequence of interdependent subtasks. Earlier subtasks introduce design goals, rules, feedback, or constraints. Later subtasks require the agent to reuse this information when producing revised recommendations. The scenarios are abstracted and do not include company-specific product data, customer information, proprietary design rules, or confidential engineering processes.

### *F. Agent Execution Pipeline*

All experimental configurations follow the same execution pipeline.

1) **Task input:** The agent receives the current task, including relevant rules, objectives, and constraints.
2) **Memory retrieval:** If memory is enabled, the agent retrieves relevant information from the selected memory system.
3) **Response generation:** The agent generates an output using the selected LLM backbone, the current task input, and any retrieved memory.
4) **Evaluation:** The output is evaluated using predefined criteria, such as task success, rule compliance, and consistency.
5) **Feedback:** Structured feedback is generated to indicate errors, constraint violations, or improvement opportunities.
6) **Memory update:** Depending on the memory system, the task input, agent output, feedback, and relevant interaction trace are stored or summarized into memory.
7) **Iteration:** The agent proceeds to the next subtask or interaction round.

This pipeline forms a Memory–Agent–Environment interaction cycle in which memory can be updated and reused across sessions.

### *G. Experimental Design*

We adopt a factorial experimental design [23] with three factors:

- **Memory system:** stateless baseline, AutoMemory/AutoDream, Hindsight, Mem0, AContext, Zep, and

TABLE I
SELECTED MEMORY SYSTEMS FOR THE EXPERIMENTAL EVALUATION

| **Memory system** | **Memory strategy** | **Role in this study** |
|---|---|---|
| Stateless baseline | No persistent memory | Used as the baseline condition. The agent processes each task using only the current input and available context, without storing or retrieving information across interactions. |
| AutoMemory and AutoDream | Built-in proprietary memory | Used to evaluate built-in memory mechanisms that support storing and reusing information across interactions. The final description will be completed based on the implementation details provided by the industrial partner. |
| Hindsight | Reflection-oriented memory | Used to evaluate whether feedback-driven reflection helps the agent improve across iterations and avoid repeated mistakes. |
| Mem0 | Managed long-term memory | Used to evaluate structured memory operations, including storing, updating, and retrieving information from previous interactions. |
| AContext | Skill-based memory | Used to evaluate whether reusable skills or learned interaction patterns can support performance across related tasks. |
| Zep | Temporal knowledge graph memory | Used to evaluate memory organized around temporal relationships and evolving knowledge across sessions. |
| Supermemory | Retrieval-based persistent memory | Used to evaluate persistent retrieval-based memory for storing and retrieving prior task information. |

TABLE II
CANDIDATE LLM BACKBONES CONSIDERED FOR EVALUATING MEMORY-ENABLED AGENTS

| **Provider** | **Candidate model(s)** | **Purpose in this study** |
|---|---|---|
| OpenAI | GPT-5.5 Pro; GPT-5.5 Thinking | Used to evaluate memory mechanisms with strong general-purpose and reasoning-oriented models. GPT-5.5 Thinking may be used for tasks requiring more specialized reasoning. |
| Google | Gemini 3.5 Flash; Gemini 3.1 Pro | Used to evaluate memory mechanisms with Google models representing fast and reasoning-capable configurations. |
| Anthropic | Claude Opus 4.7 | Used to evaluate memory mechanisms with a model family known for long-form reasoning and instruction-following behavior. |
| Meta | Llama 4 Behemoth; Llama 4 Scout | Used to evaluate memory mechanisms with open-weight or locally deployable models. Llama 4 Scout may be considered for long-context task settings. |
| DeepSeek | DeepSeek-V4 Pro; DeepSeek-R1 | Used to evaluate memory mechanisms with reasoning-focused and potentially locally deployable model configurations. |

Supermemory.

- **LLM backbone:** OpenAI, Google, Anthropic, Meta, and DeepSeek models.
- **Iteration depth:** sequential interaction rounds within each task scenario.

Each configuration is evaluated across multiple task instances. Where stochastic model settings are used, repeated runs are conducted to reduce the influence of random variation. For iterative tasks, the order of interactions is preserved because later tasks can depend on earlier decisions, feedback, or constraints.

### *H. Dependent Variables and Metrics*

We evaluate both task performance and memory behavior. The dependent variables are aligned with the research questions.

- **Task success rate** measures the proportion of tasks or subtasks completed correctly.
- **Rule compliance** measures the degree to which the agent output satisfies the given DFx rules, constraints, or benchmark requirements.
- **Progress score** captures partial task completion across subtasks:

$$PS = \frac{\#\text{correct subtasks}}{\#\text{total subtasks}}$$

- **Improvement over time** measures whether agent performance improves across interaction rounds. This can be calculated through the slope of performance metrics across iterations or through relative improvement between early and later interactions.
- **Consistency** measures whether the agent avoids contradictions across related interactions. In DFx scenarios, this includes consistency with earlier recommendations, feedback, accepted decisions, and active constraints.
- **Efficiency** measures the computational cost of each configuration, including latency, token usage, number of retrieved memory entries, and memory storage size where applicable.
- **Memory utility** measures whether memory contributes to task performance. It includes the relevance of retrieved memory, correctness of retrieved information, use of retrieved memory in the output, and performance gain relative to the stateless baseline.

### I. Data Collection

For each experimental run, we collect task-level and memory-level data. Task-level data include the task input, agent output, evaluation feedback, final score, model configuration, and execution metadata. Memory-level data include memory writes, retrieved memory entries, memory updates, retrieval timing, and whether retrieved information was used in the final output.

For the DFx scenarios, domain experts from Etteplan contribute to the task design and review a subset of agent outputs. Their input is used to ensure that the scenarios reflect realistic engineering concerns while remaining abstracted and non-confidential.

### J. Quantitative Analysis

Quantitative analysis compares agent performance across memory systems, LLM backbones, and iteration depths. We first compute descriptive statistics for each configuration, including task success rate, rule compliance, progress score, consistency, efficiency, and memory utility.

To analyze learning behavior, we examine performance trends across iterations. This includes the slope of performance over time, relative improvement between early and late iterations, and stability across repeated runs. Pairwise comparisons are used to quantify differences between memory-enabled configurations and the stateless baseline.

Where the number of observations is sufficient and assumptions are satisfied, we use statistical tests such as ANOVA to examine the main and interaction effects of memory system and LLM backbone [23]. Post-hoc tests, such as Tukey's HSD, may be used to identify differences between configurations [24]. Effect sizes are reported where applicable to assess the practical importance of observed differences [25].

### K. Qualitative Analysis

Qualitative analysis is used to explain how and why memory affects agent behavior. We manually inspect selected successful and unsuccessful task executions, focusing on how agents retrieve prior information, incorporate feedback, and adapt their responses across iterations. This complements the quantitative analysis and supports a mixed-methods interpretation of the results [26].

Failure cases are categorized according to the stage of the memory process in which the problem occurs. The categories include missing memory write, irrelevant memory write, failed retrieval, retrieval of incorrect or outdated information, incorrect use of retrieved information, over-reliance on prior feedback, and inconsistency across iterations. This analysis follows an open coding style similar to qualitative thematic analysis [27] and helps explain performance differences that may not be visible from aggregate scores alone.

### L. Industrial Expert Evaluation

To complement benchmark-based evaluation, Etteplan domain experts evaluate a subset of outputs from the DFx scenarios. The expert evaluation focuses on practical correctness, usefulness of the solution, adherence to DFx constraints, clarity of the agent's reasoning, and applicability of the output in engineering workflows.

This evaluation provides an industrial perspective on whether memory-enabled agents produce outputs that are not only technically correct, but also useful and interpretable in realistic engineering settings.

### M. Reproducibility and Confidentiality

For public benchmark evaluation, we document the prompts, model configurations, memory settings, task inputs, evaluation scripts, and scoring criteria where possible. For DFx scenarios, we provide abstracted task descriptions and evaluation criteria without exposing confidential information. All examples used in the paper are illustrative and do not disclose company-specific product data, customer information, proprietary design rules, or confidential engineering processes.

## REFERENCES


[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] OpenAI, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *The eleventh international conference on learning representations*, 2022.

[4] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," *Advances in neural information processing systems*, vol. 36, pp. 68 539–68 551, 2023.

[5] E. Bandara, R. Gore, S. Shetty, S. Rajapakse, I. Kularathna, P. Karunarathna, R. Mukkamala, P. Foytik, S. H. Bouk, A. Rahman *et al.*, "A practical guide to agentic ai transition in organizations," *arXiv preprint arXiv:2602.10122*, 2026.

[6] Z. He, Y. Wang, C. Zhi, Y. Hu, T.-P. Chen, L. Yin, Z. Chen, T. A. Wu, S. Ouyang, Z. Wang, J. Pei, J. McAuley, Y. Choi, and A. Pentland, "Benchmarking agent memory in interdependent multi-session agentic tasks," 2026. [Online]. Available: https://arxiv.org/abs/2602.16313

[7] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," 2023. [Online]. Available: https://arxiv.org/abs/2310.08560

[8] N. Shinn, F. Cassano, B. Labash, D. Gopinath *et al.*, "Reflexion: Language agents with verbal reinforcement learning," *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[9] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[10] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global: A graph rag approach to query-focused summarization," 2024. [Online]. Available: https://arxiv.org/abs/2404.16130

[11] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "Memorybank: Enhancing large language models with long-term memory," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 724–19 731. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/29946

[12] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang, "A-mem: Agentic memory for llm agents," in *Advances in Neural Information Processing Systems*, 2025. [Online]. Available: https://papers.neurips.cc/paper_files/paper/2025/hash/19909c36f51abc4856b4560aff3d36d6-Abstract-Conference.html

[13] Y. Wang and X. Chen, "Mirix: Multi-agent memory system for llm-based agents," *arXiv preprint arXiv:2507.07957*, 2025. [Online]. Available: https://arxiv.org/abs/2507.07957

[14] S. Yan, X. Yang, Z. Huang, E. Nie, Z. Ding, Z. Li, X. Ma, H. Schütze, V. Tresp, and Y. Ma, "Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning," *arXiv preprint arXiv:2508.19828*, 2025. [Online]. Available: https://arxiv.org/abs/2508.19828

[15] G. Sarch, Y. Wu, M. J. Tarr, and K. Fragkiadaki, "Open-ended instructable embodied agents with memory-augmented large language models," in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Singapore: Association for Computational Linguistics, 2023, pp. 3468–3500. [Online]. Available: https://aclanthology.org/2023.findings-emnlp.226/

[16] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of llm agents," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 13 851–13 870. [Online]. Available: https://aclanthology.org/2024.acl-long.747/

[17] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "Longmemeval: Benchmarking chat assistants on long-term interactive memory," in *Proceedings of the Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html

[18] H. Tan, Z. Zhang, C. Ma, X. Chen, Q. Dai, and Z. Dong, "MemBench: Towards more comprehensive evaluation on the memory of LLM-based agents," in *Findings of the Association for Computational Linguistics: ACL 2025*. Vienna, Austria: Association for Computational Linguistics, 2025, pp. 19 336–19 352. [Online]. Available: https://aclanthology.org/2025.findings-acl.989/

[19] Y. Hu, Y. Wang, and J. McAuley, "Evaluating memory in llm agents via incremental multi-turn interactions," in *Proceedings of the Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: https://openreview.net/forum?id=DT7JyQC3MR

[20] D. Castillo-Bolado, J. Davidson, F. Gray, and M. Rosa, "Beyond prompts: Dynamic conversational benchmarking of large language models," in *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/4aedf0cba303537fcb6cf948bb41b2df-Abstract-Datasets_and_Benchmarks_Track.html

[21] Microsoft, "Microsoft agent framework documentation," https://learn.microsoft.com/en-us/agent-framework/, 2026, accessed: 2026-05-28.

[22] ——, "Microsoft agent framework," https://github.com/microsoft/agent-framework, 2026, accessed: 2026-05-28.

[23] D. C. Montgomery, *Design and Analysis of Experiments*, 9th ed. John Wiley & Sons, 2017.

[24] J. W. Tukey, "Comparing individual means in the analysis of variance," *Biometrics*, vol. 5, no. 2, pp. 99–114, 1949.

[25] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Lawrence Erlbaum Associates, 1988.

[26] J. W. Creswell and V. L. Plano Clark, *Designing and Conducting Mixed Methods Research*, 3rd ed. SAGE Publications, 2018.

[27] V. Braun and V. Clarke, "Using thematic analysis in psychology," *Qualitative Research in Psychology*, vol. 3, no. 2, pp. 77–101, 2006.